Data Visualization Mastery Lab

GROUP 1: Single Variables and Categories (Foundations)

Your goal in this group is to understand the distribution of single data points and compare different categories.

Part 1: Distribution Analysis

Before comparing variables, you must understand how individual variables are distributed.

Instructions for Part 1:

1. Choose one continuous numerical column from your dataset (e.g., `Global_Sales` for Video Games, `Speed` for Pokemon, or `tenure` for Churn).
2. Create a **Histogram** of this column using Seaborn.
3. Enable the KDE (Kernel Density Estimate) curve on the histogram to see the smooth trend line.
4. Change the color of the histogram to a color of your choice (e.g., purple or green).
5. Add a descriptive Title and label the X-axis.

Part 2: Categorical Comparisons

Let's see which categories dominate the dataset.

Instructions for Part 2:

1. Choose a categorical column (e.g., `Genre` for Video Games, `Type 1` for Pokemon, or `Contract` for Churn).
2. Create a **Countplot** using Seaborn to show the number of entries for each category.
3. *Challenge:* If your x-axis labels are overlapping and hard to read, use Matplotlib to rotate the x-ticks by 45 or 90 degrees!
4. Choose a continuous column and a categorical column. Create a **Boxplot** to show the distribution and outliers of the continuous variable across the different categories.

GROUP 2: Relationships and Correlations

Your goal in this group is to figure out how different columns interact with one another.

Part 1: Relational Scatterplots

Do higher stats mean higher overall power? Do higher monthly charges correlate with higher total charges?

Instructions for Part 1:

1. Pick two numeric columns that you suspect might be related (e.g., `NA_Sales` vs. `EU_Sales`, `Attack` vs. `Defense`, or `MonthlyCharges` vs. `TotalCharges`).
2. Create a **Scatterplot** using Seaborn with these two columns on the X and Y axes.
3. Use the `hue` parameter to color-code the dots based on a third, categorical column (e.g., color by `Genre`, `Legendary` status, or `Churn`).
4. Adjust the `alpha` (transparency) of the dots so you can see where they overlap heavily.

Part 2: The Heatmap

Let's look at the mathematical correlation of every number in the dataset at once.

Instructions for Part 2:

1. Isolate only the numerical columns in your DataFrame (drop text columns temporarily or use pandas' numeric-only selection). *(Note for Option C: You may need to convert `TotalCharges` to numeric first!)*
2. Calculate the correlation matrix using `.corr()`.
3. Pass this matrix into a Seaborn **Heatmap**.
4. Set `annot=True` to display the exact correlation numbers inside the colored boxes.
5. Change the color palette using the `cmap` parameter (try `'coolwarm'` or `'viridis'`).

GROUP 3: Advanced Storytelling and Export

Your goal in this group is to build a professional, multi-chart dashboard and annotate specific insights.

Part 1: Subplot Grid Layout

Combine multiple insights into one single image.

Instructions for Part 1:

1. Initialize a Matplotlib figure with a specific size (e.g., 12 inches wide by 5 inches tall).
2. Set up a grid with 1 Row and 2 Columns using `plt.subplot()`.
3. In the **first subplot (left)**, recreate your Scatterplot from Part 4.
4. In the **second subplot (right)**, create a standard Barplot (using `sns.barplot`) showing the average of a numeric column grouped by a categorical column.
5. Use `plt.tight_layout()` at the end so the graphs do not overlap.

Part 2: Annotations and Polish

Draw the viewer's eye to a specific data point.

Instructions for Part 1:

1. Pick one of your previous graphs (or create a simple Line/Scatter plot).
2. Use pandas to programmatically find the absolute maximum value in the Y-axis column (e.g., the highest-selling game, the Pokemon with the highest HP, or the customer with the highest MonthlyCharge).
3. Use Matplotlib's `plt.text()` or `plt.annotate()` feature to place a text label exactly at that coordinate.
4. The text should say something like "Record High!" or name the specific data point.
5. Ensure your final plot has a main Title, X/Y labels, and a clear legend.

Part 3: Final Export

Save your visual insights to share in a presentation.

Instructions for Part 3:

1. Before calling `plt.show()`, use Matplotlib's `plt.savefig()` method.
2. Save your final polished chart (from Part 7) as an image file named `final_visualization.png`.

3. *Bonus:* Pass the argument `dpi=300` into the save function to ensure the image is exported in high-definition!